

Vector in C++ STL

Last Updated : 23 Jul, 2025



C++ **vector** is a dynamic array that stores collection of elements same type in contiguous memory. It has the ability to resize itself automatically when an element is inserted or deleted.

Create a Vector

Before creating a vector, we must know that a vector is defined as the **std::vector** class template in the **<vector>** header file.

```
vector<T> v;
```



where **T** is the data type of elements and **v** is the name assigned to the vector.

Now we are creating an instance of `std::vector` class. This requires us to provide the type of elements as template parameter.

Example:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {

    // Creating an empty vector
    vector<int> v1;

    return 0;
}
```



We can also provide the values to be stored in the vector inside **{}** curly braces. This process is called **initialization**.

```
#include <iostream>
#include <vector>
using namespace std;
void printVector(vector<int>& v) {
    for (auto x: v) {
```



```

        cout << x << " ";
    }
    cout << endl;
}
int main() {

    // Creating a vector of 5 elements from
    // initializer list
    vector<int> v1 = {1, 4, 2, 3, 5};

    // Creating a vector of 5 elements with
    // default value
    vector<int> v2(5, 9);

    printVector(v1);
    printVector(v2);
    return 0;
}

```

Output

```

1 4 2 3 5
9 9 9 9 9

```

In the above example,

- Statement **vector<int> v1 = {1, 4, 2, 3, 5}** initializes a vector with given values.
- Statement **vector<int> v2(5, 9)** creates a vector of size 5 where each element initialized to 9.

More ways to declare and initialize vectors are discussed in this article - [8 Ways to Initialize Vector in C++](#)

Insert Elements

An element can be inserted into a vector using **vector insert()** method which takes linear time. But for the insertion at the end, the **vector push_back()** method can be used. It is much faster, **taking only constant time**.

```

#include <iostream>
#include <vector>

```



```
using namespace std;

int main() {
    vector<char> v = {'a', 'f', 'd'};

    // Inserting 'z' at the back
    v.push_back('z');

    // Inserting 'c' at index 1
    v.insert(v.begin() + 1, 'c');

    for (int i = 0; i < v.size(); i++)
        cout << v[i] << " ";
    return 0;
}
```

Output

```
a c f d z
```

More ways to insert an element in the vector are discussed in the article - [Different Ways to Add Elements in a Vector](#)

Access or Update Elements

Just like arrays, vector elements can be accessed using their index inside the `[]` [subscript operator](#). While accessing elements, we can also update the value of that index using [assignment operator](#) `=`. The `[]` **subscript operator** doesn't check whether the given index exists in the vector or not. So, there is another member method [vector at\(\)](#) for safely accessing or updating elements.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<char> v = {'a', 'c', 'f', 'd', 'z'};

    // Accessing and printing values
    cout << v[3] << endl;
    cout << v.at(2) << endl;
```

```
// Updating values using indexes 3 and 2
v[3] = 'D';
v.at(2) = 'F';

cout << v[3] << endl;
cout << v.at(2);
return 0;
}
```

Output

```
d
f
D
F
```

To know more about accessing and updating vector elements, refer to the article - [Different Ways to Access](#) and [Update](#) Elements in Vector.

Find Vector Size

One of the common problems with arrays was to keep a separate variable to store the size information. Vector provides the solution to this problem by providing [size\(\)](#) method.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<char> v = {'a', 'c', 'f', 'd', 'z'};

    // Finding size
    cout << v.size();

    return 0;
}
```

Output

Traverse Vector

Vector in C++ can be traversed using indexes in a loop. The indexes start from 0 and go up to vector size - 1. To iterate through this range, we can use a loop and determine the size of the vector using the [vector size\(\)](#) method.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<char> v = {'a', 'c', 'f', 'd', 'z'};

    // Traversing vector using range based for loop
    for (int i = 0; i < v.size(); i++)
        cout << v[i] << " ";
    return 0;
}
```

Output

```
a c f d z
```

We can also use a range-based loop for simple traversal. More ways to traverse vectors are discussed in this article - [Different Ways to Iterate Through Vector](#)

Delete Elements

An element can be deleted from a vector using [vector erase\(\)](#) but this method needs iterator to the element to be deleted. If only the value of the element is known, then **find()** function is used to find the position of this element.

For the deletion at the end, the [vector pop_back\(\)](#) method can be used, and it is much faster, taking only constant time.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
```

```
int main() {
    vector<char> v = {'a', 'c', 'f', 'd', 'z'};

    // Deleting last element 'z'
    v.pop_back();

    // Deleting element 'f'
    v.erase(find(v.begin(), v.end(), 'f'));

    for (int i = 0; i < v.size(); i++) {
        cout << v[i] << " ";
    }
    return 0;
}
```

Output

```
a c d
```

To know more about the deletion of an element in the vector, refer to this article - [Different Ways to Remove Elements from Vector](#)

Other Operations

Vector is one of the most frequently used containers in C++. It is used in many situations for different purposes. The following examples aim to help you master vector operations beyond the basics.

- [Check if a Vector is Empty](#)
- [Find the Size of a Vector](#)
- [Resize a Vector](#)
- [Reverse a Vector](#)
- [Sort a Vector in Ascending Order](#)
- [Sort a Vector in Descending Order](#)
- [Different Ways to Copy a Vector](#)
- [Swap Two Vectors](#)
- [Remove Duplicates from a Vector](#)

Time Complexity

The below table lists the time complexity of the above operations on a vector:

Operation	Time Complexity
Insert an element at the end	O(1) (amortized)
Insert an element somewhere in the middle	O(n)
Delete an element from the end	O(1)
Delete an element from somewhere in the middle	O(n)
Access an element by index	O(1)
Traverse the vector	O(n)
Find an element by value	O(n)

Passing Vector to Functions

Vectors can be passed to a function as arguments just like any other variable in C++. But it is recommended to pass the vector by reference so as to avoid the copying of all elements which can be expensive if the vector is large. Refer to this article to know more - [Passing Vector to a Function](#)

Internal Working of Vector

Vector internal working is very interesting and useful to select and optimize its usage. Understanding the internal memory management also helps in modifying the default mechanism of vector to suits our needs. Refer to this article to know more - [Internal Working of Vector](#)

Multidimensional Vectors

Just like arrays, we can also create multidimensional vectors in C++. Each element of multidimensional vector can be visualized as the collection of vectors with

dimension one less than the current vector. For example, 2D vectors are the collection of 1D vectors, while 3D vectors are the collection of 2D vectors and so on.

With the addition of each dimension, the complexity of operations on the vectors also increases.

Refer to this article to know more - [Multidimensional Vectors in C++](#)

Member Functions of Vector

Following is the list of all member functions of `std::vector` class in C++:

Function	Description
<code>push_back()</code>	Adds an element to the end of the vector.
<code>pop_back()</code>	Removes the last element of the vector.
<code>size()</code>	Returns the number of elements in the vector.
<code>max_size()</code>	Returns the maximum number of elements that the vector can hold.
<code>resize()</code>	Changes the size of the vector.
<code>empty()</code>	Checks if the vector is empty.
<code>operator[]</code>	Accesses the element at a specific position.
<code>at()</code>	Accesses the element at a specific position, with bounds checking.
<code>front()</code>	Accesses the first element of the vector.
<code>back()</code>	Accesses the last element of the vector.
<code>begin()</code>	Returns an iterator pointing to the first element of the vector.

Function	Description
<u>end()</u>	Returns an iterator pointing to the past-the-end element of the vector.
<u>rbegin()</u>	Returns a reverse iterator pointing to the last element of the vector.
<u>rend()</u>	Returns a reverse iterator pointing to the element preceding the first element of the vector.
<u>cbegin</u>	Returns const_iterator to beginning
<u>cend</u>	Returns const_iterator to end
<u>crbegin</u>	Returns const_reverse_iterator to reverse beginning
<u>crend</u>	Returns const_reverse_iterator to reverse end
<u>insert()</u>	Inserts elements at a specific position in the vector.
<u>erase()</u>	Removes elements from a specific position or range in the vector.
<u>swap()</u>	Swaps the contents of the vector with those of another vector.
<u>clear()</u>	Removes all elements from the vector.
<u>emplace()</u>	Constructs and inserts an element in the vector.
<u>emplace_back()</u>	Constructs and inserts an element at the end of the vector.
<u>assign()</u>	Assigns new values to the vector elements by replacing old ones.
<u>capacity()</u>	Returns the size of the storage space currently allocated to the vector.

Function	Description
<code>reserve()</code>	Requests that the vector capacity be at least enough to contain a specified number of elements.
<code>shrink_to_fit()</code>	Reduces memory usage by freeing unused space.
<code>data()</code>	Returns a direct pointer to the memory array used internally by the vector to store its owned elements.
<code>get_allocator()</code>	Returns a copy of the allocator object associated with the vector.

Vector in C++

[Visit Course ↗](#)
[Comment](#)
[More info ▼](#)
[Advertise with us](#)
[Next Article >](#)
[8 Ways to Initialize Vector in C++](#)

Similar Reads

Vector in C++ STL

C++ vector is a dynamic array that stores collection of elements same type in contiguous memory. It has the ability to resize itself automatically when an element is inserted or deleted. Create a Vector Before...

🕒 7 min read

8 Ways to Initialize Vector in C++

Initializing a vector means assigning some initial values to the `std::vector` elements. In this article, we will learn 8 different ways to initialize a vector in C++. Table of Content Using Initializer List One by One...

🕒 5 min read

Commonly Used Methods



Other Member Methods



2D Vector in C++

A 2D vector is a vector of the vectors i.e. each element is a vector in itself. It can be visualised as a matrix where each inner vector represents a row, and the number of rows represents the maximum columns. A...

🕒 7 min read

Passing Vector to a Function in C++

To perform operations on vector belonging to one function inside other function, we pass this vector to the function as arguments during the function call.C++ provides three methods to pass a vector to a...

🕒 5 min read

How Does a Vector Internally Works in C++?

In C++, a vector is a dynamic array that can resize itself when more elements are added or deleted. So, one may ask that how a vector internally works to achieve its dynamic resizing capability while...

🕒 5 min read

How to implement our own Vector Class in C++?

The given task is to implement a class in C++ which behaves just like the Vector class.Vectors are the same as dynamic arrays with the ability to resize themselves automatically when an element is inserted...

🕒 5 min read

Advantages of Vector Over Array in C++

In C++, both vectors and arrays are used to store collections of elements, but vector offers significant advantages over arrays in terms of flexibility, functionality, and ease of use. This article explores the...

🕒 3 min read

Common Vector Programs



Do Not Sell or Share My Personal Information